Modelo Pseudotsuga menziesii

Viacheslav Shalsiko

2026-03-20

Preparar el entorno

```
library(sp)
```

```
## Warning: package 'sp' was built under R version 4.5.2
```

```
#library(rgdal)
library(raster)
```

```
## Warning: package 'raster' was built under R version 4.5.2
```

```
library(dismo)
```

```
## Warning: package 'dismo' was built under R version 4.5.2
```

```
library(rJava)
```

```
## Warning: package 'rJava' was built under R version 4.5.2
```

```
library(rworldmap)
```

```
## Warning: package 'rworldmap' was built under R version 4.5.2
```

```
## ### Welcome to rworldmap ###
```

```
## For a short introduction type :   vignette('rworldmap')
```

```
library(rworldxtra)
```

```
## Warning: package 'rworldxtra' was built under R version 4.5.2
```

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
## 
## Matrix products: default
##   LAPACK version 3.12.1
## 
## locale:
## [1] LC_COLLATE=Spanish_Mexico.utf8  LC_CTYPE=Spanish_Mexico.utf8   
## [3] LC_MONETARY=Spanish_Mexico.utf8 LC_NUMERIC=C                   
## [5] LC_TIME=Spanish_Mexico.utf8    
## 
## time zone: America/Mexico_City
## tzcode source: internal
## 
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base     
## 
## other attached packages:
## [1] rworldxtra_1.01 rworldmap_1.3-8 rJava_1.0-14    dismo_1.3-16   
## [5] raster_3.6-32   sp_2.2-0       
## 
## loaded via a namespace (and not attached):
##  [1] terra_1.8-60       cli_3.6.5          knitr_1.50         rlang_1.1.6       
##  [5] xfun_0.52          dotCall64_1.2      jsonlite_2.0.0     fields_17.1       
##  [9] htmltools_0.5.8.1  sass_0.4.10        rmarkdown_2.29     grid_4.5.1        
## [13] evaluate_1.0.4     jquerylib_0.1.4    fastmap_1.2.0      yaml_2.3.10       
## [17] lifecycle_1.0.4    compiler_4.5.1     codetools_0.2-20   RColorBrewer_1.1-3
## [21] Rcpp_1.1.0         rstudioapi_0.18.0  maps_3.4.3         lattice_0.22-7    
## [25] digest_0.6.37      viridisLite_0.4.2  R6_2.6.1           spam_2.11-3       
## [29] bslib_0.9.0        tools_4.5.1        cachem_1.1.0
```

Cargar puntos de presencia

```
ocurrencias <- read.delim("ocurrence/occurrence.txt")
dim(ocurrencias)
```

```
## [1] 581 257
```

```
str(ocurrencias[,c("species","decimalLongitude","decimalLatitude")])
```

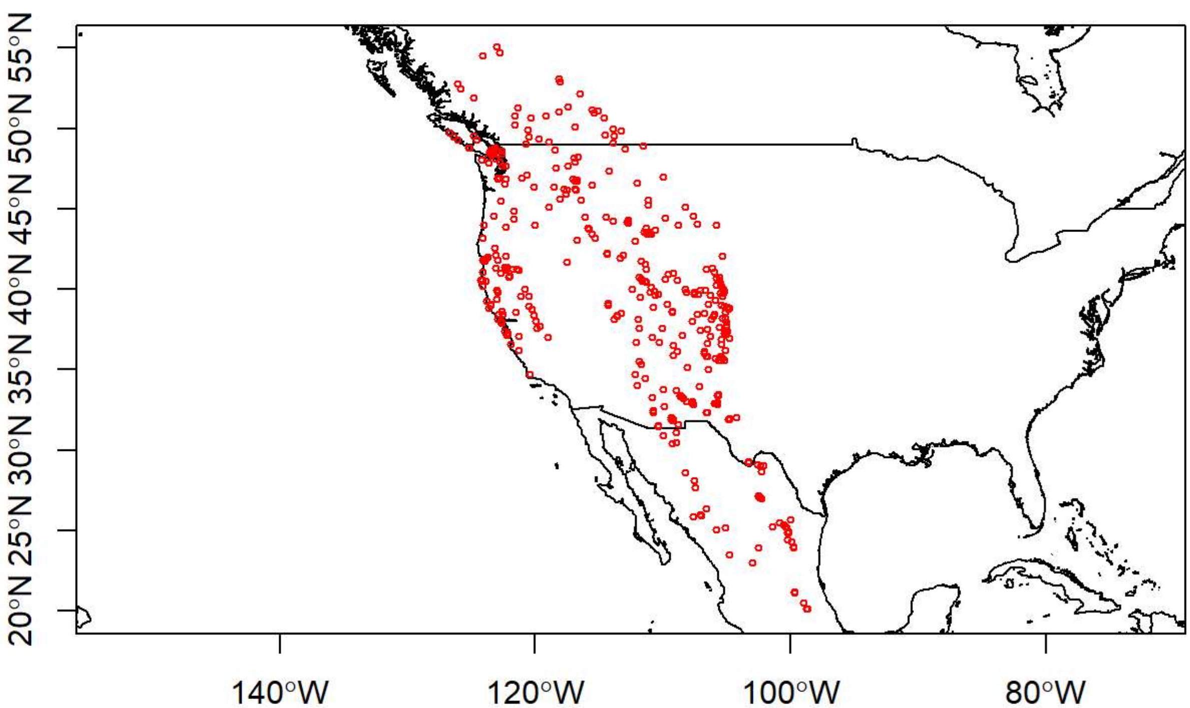
```
## 'data.frame':    581 obs. of  3 variables:
##  $ species         : chr  "Pseudotsuga menziesii" "Pseudotsuga menziesii" "Pseudotsuga menzie
sii" "Pseudotsuga menziesii" ...
##  $ decimalLongitude: num  -108 -107 -124 -106 -107 ...
##  $ decimalLatitude : num  33 32.4 40.6 33.4 32.3 ...
```

```
ocurrencias <- ocurrencias[,c("species","decimalLongitude","decimalLatitude")]
ocurrencias$species <- as.factor(ocurrencias$species)
head(ocurrencias)
```

```
##                  species decimalLongitude decimalLatitude
## 1 Pseudotsuga menziesii         -107.7388        32.97971
## 2 Pseudotsuga menziesii         -106.5688        32.35220
## 3 Pseudotsuga menziesii         -124.2571        40.56730
## 4 Pseudotsuga menziesii         -105.6267        33.37847
## 5 Pseudotsuga menziesii         -106.5598        32.34767
## 6 Pseudotsuga menziesii         -108.4573        33.27496
```

Visualizar Puntos de presencia

```
mundo <- getMap(resolution = "high")
plot(mundo, axes = TRUE, xlim = c(-130,-95), ylim = c(20,55))
points(ocurrencias$decimalLongitude, ocurrencias$decimalLatitude, cex = 0.5, col = "red")
```



Cargar datos ambientales

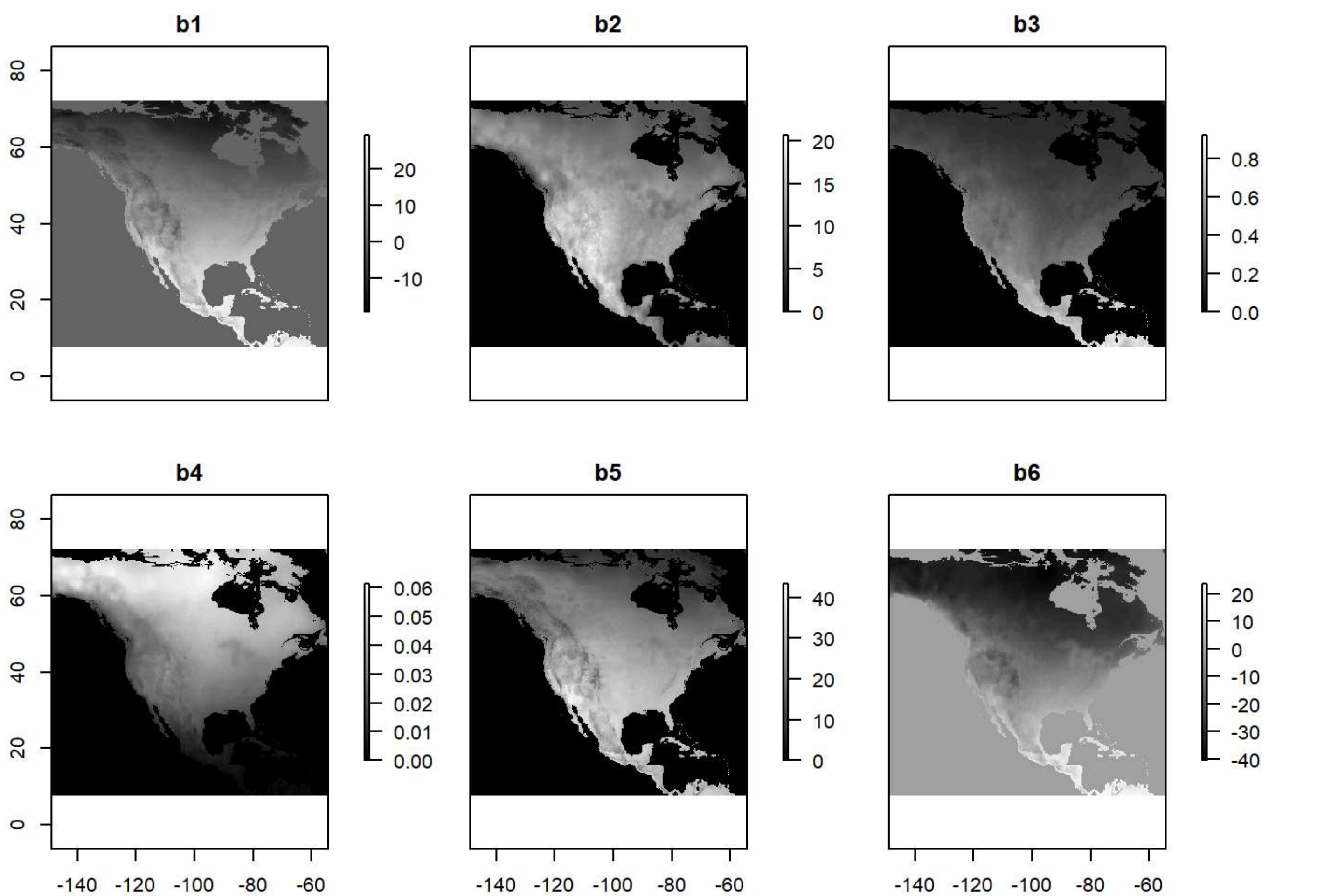
```
bio_vars <- stack("Bioclim/bioclim_40var_ok_recorte.tif")
```

```
## Warning: [minmax] min and max values not available for all layers. See
## 'setMinMax' or 'global'
```

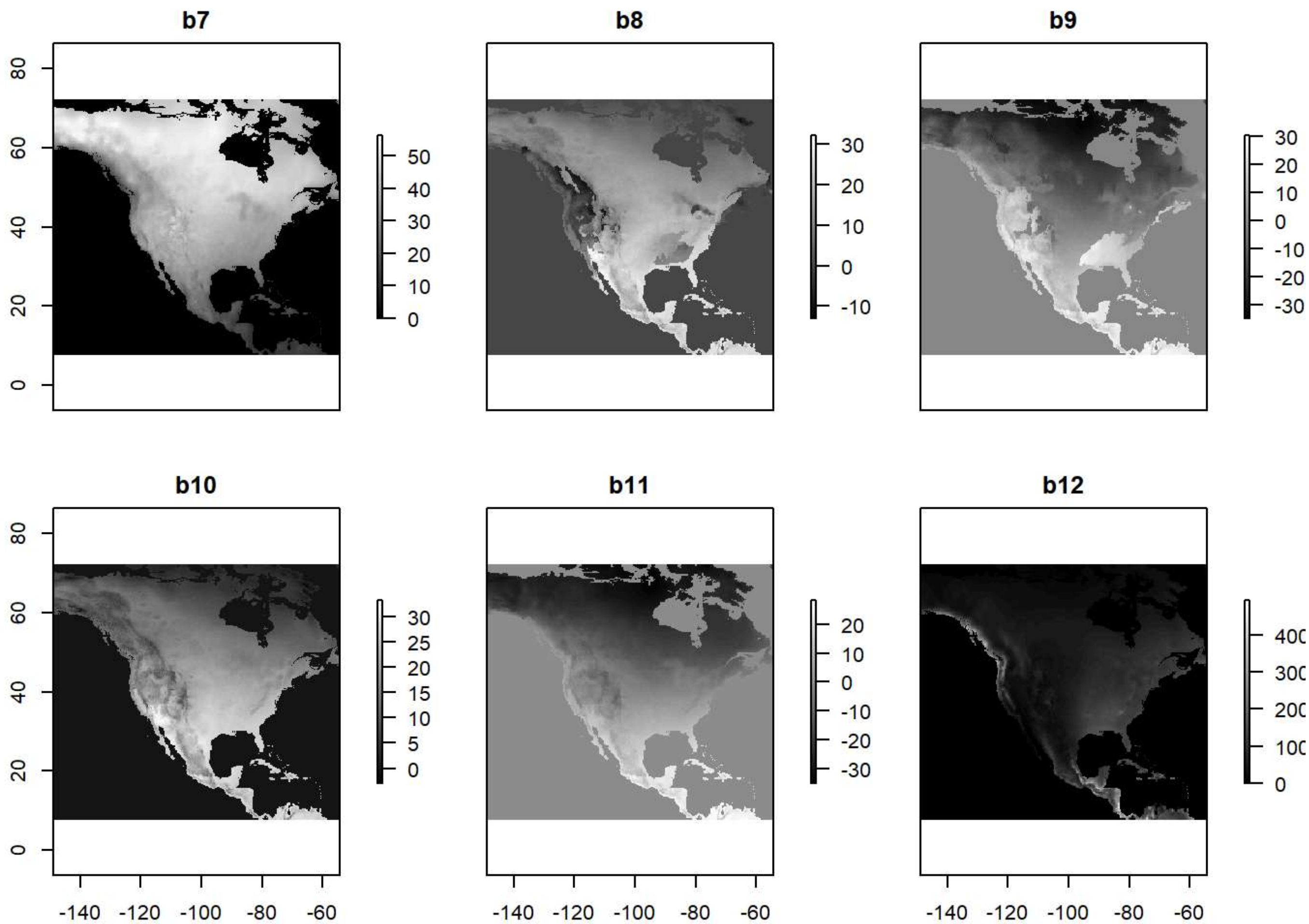
```
names(bio_vars) <- paste('b',1:40,sep='')
bio_vars
```

```
## class       : RasterStack
## dimensions : 388, 567, 219996, 40  (nrow, ncol, ncell, nlayers)
## resolution : 0.1666667, 0.1666656  (x, y)
## extent      : -148.8117, -54.31171, 7.687657, 72.3539  (xmin, xmax, ymin, ymax)
## crs         : +proj=longlat +datum=WGS84 +no_defs
## names       : b1, b2, b3, b4, b5, b6, b7, b8, b9, b10, b11, b12, b13, b14, b15, ...
```

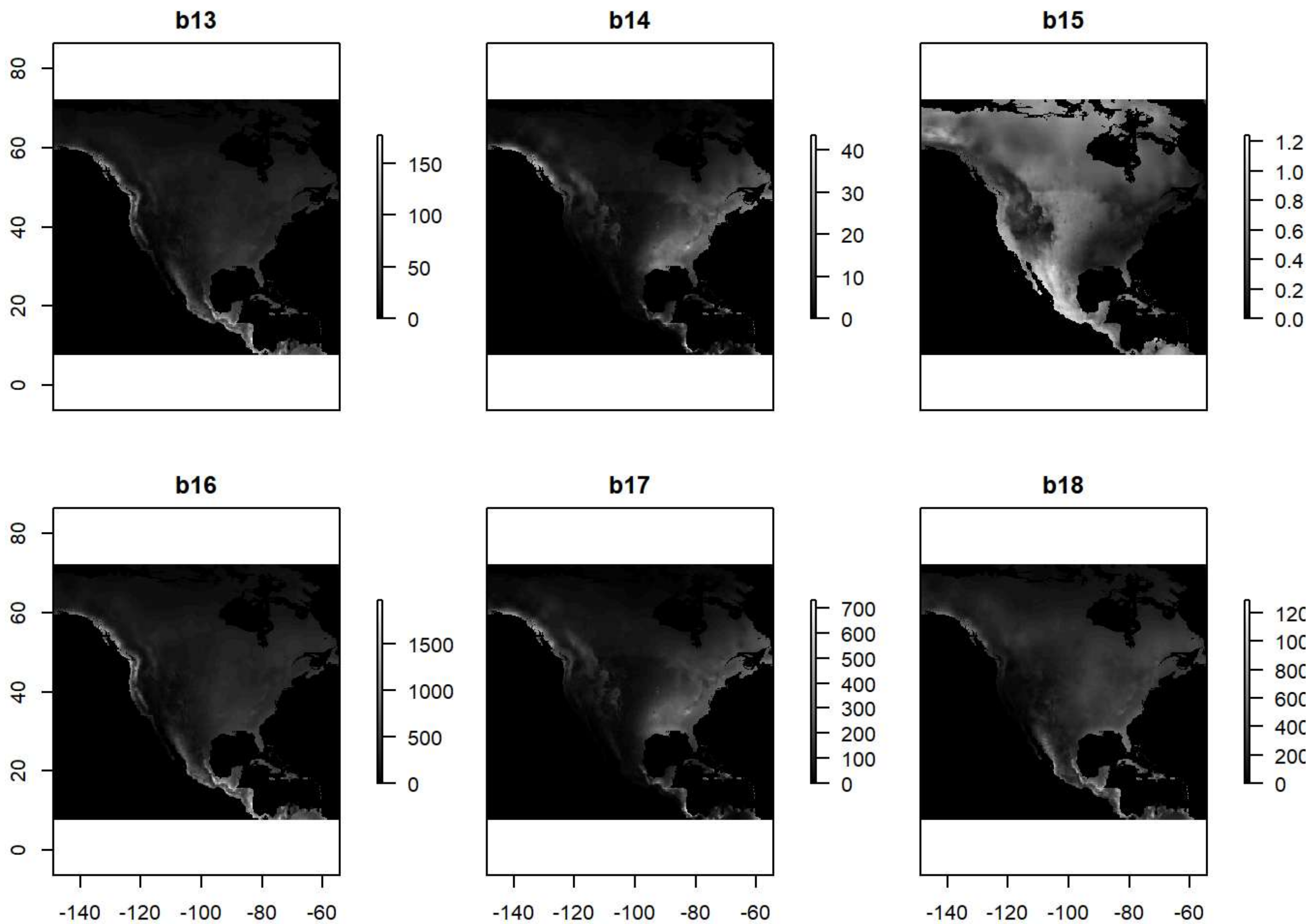
```
plot(bio_vars, c(1:6), col = grey(1:100/100), nc=3)
```



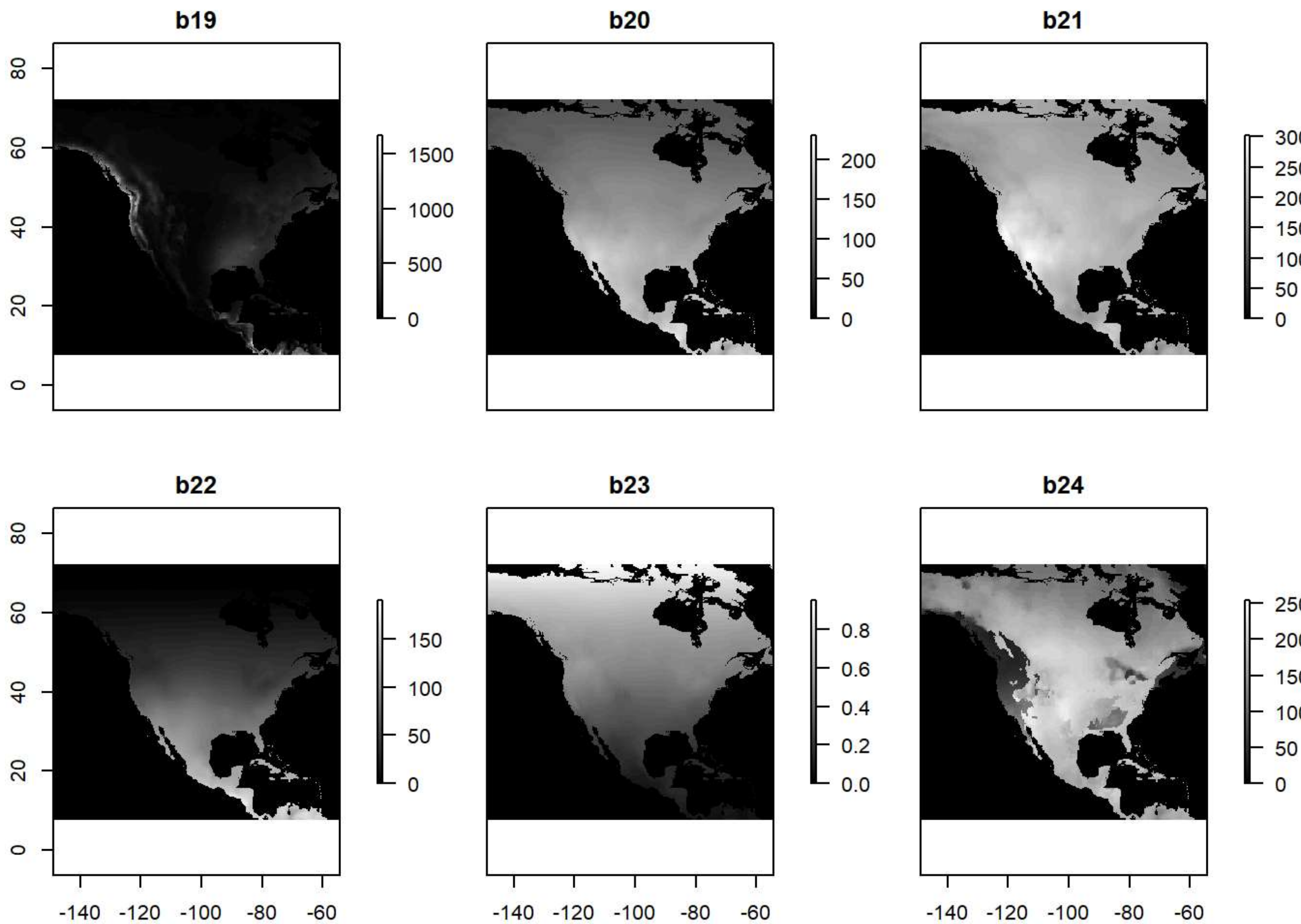

```
plot(bio_vars, c(7:12), col = grey(1:100/100), nc=3)
```



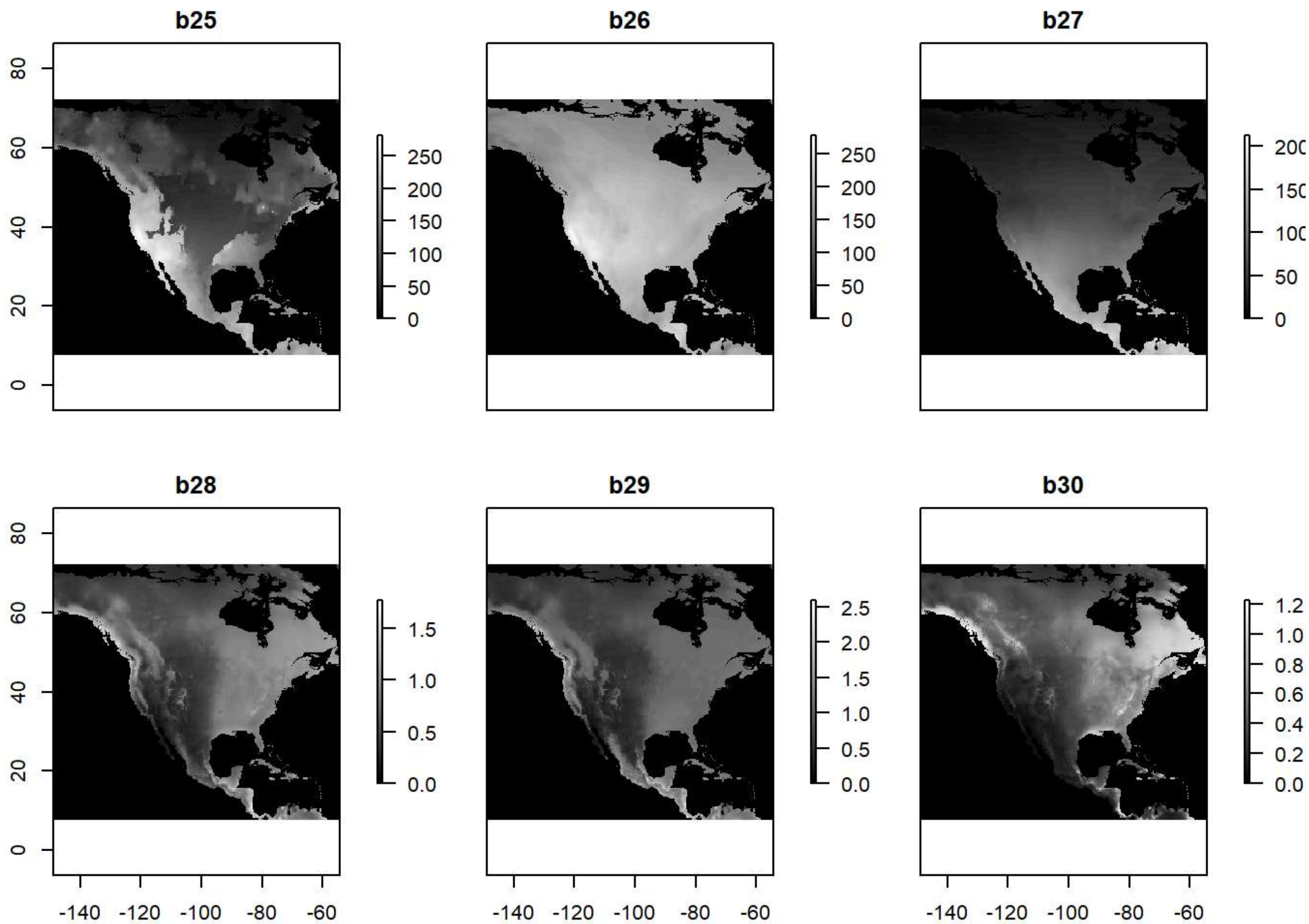

```
plot(bio_vars, c(13:18), col = grey(1:100/100), nc=3)
```



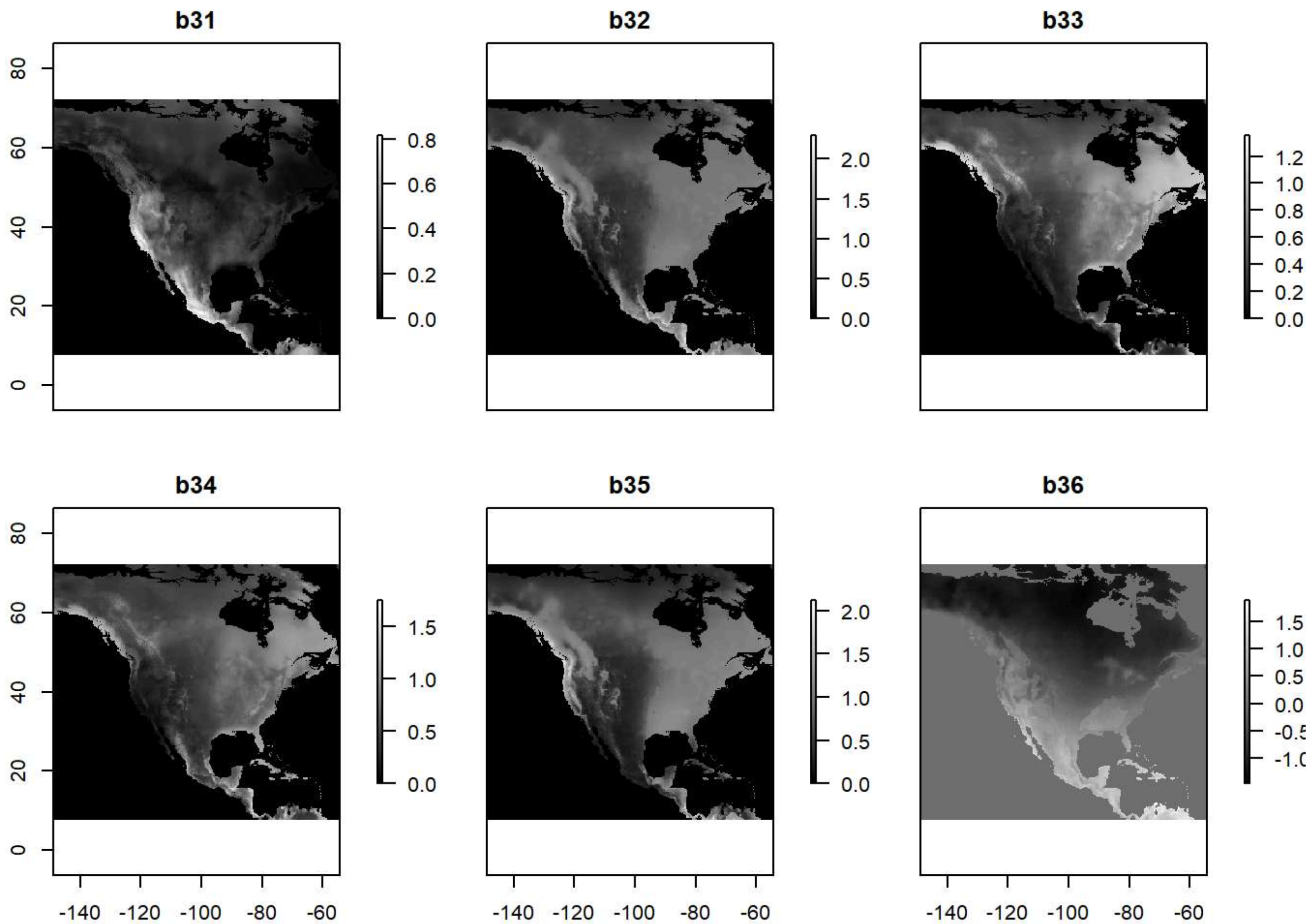

```
plot(bio_vars, c(19:24), col = grey(1:100/100), nc=3)
```



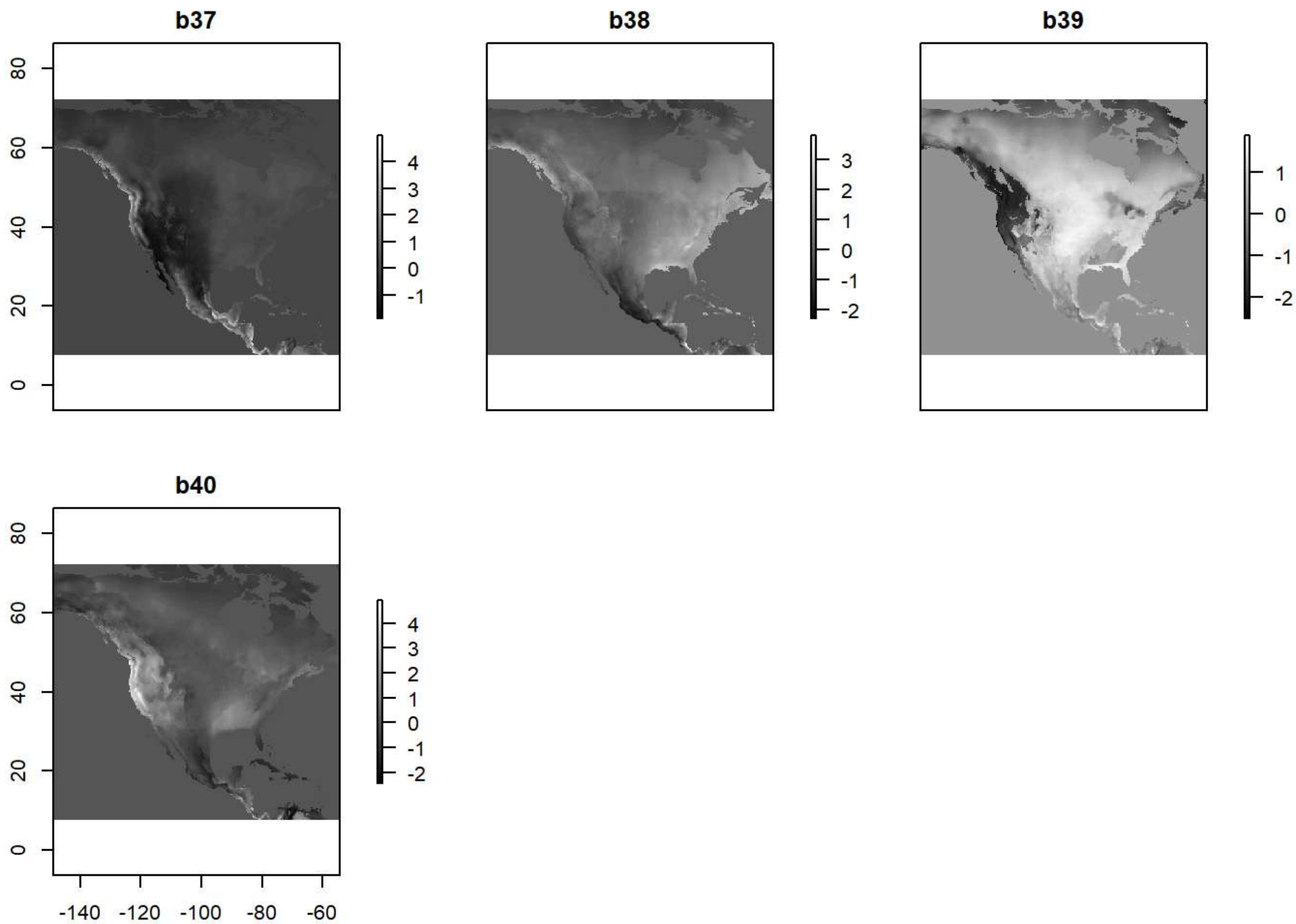

```
plot(bio_vars, c(25:30), col = grey(1:100/100), nc=3)
```



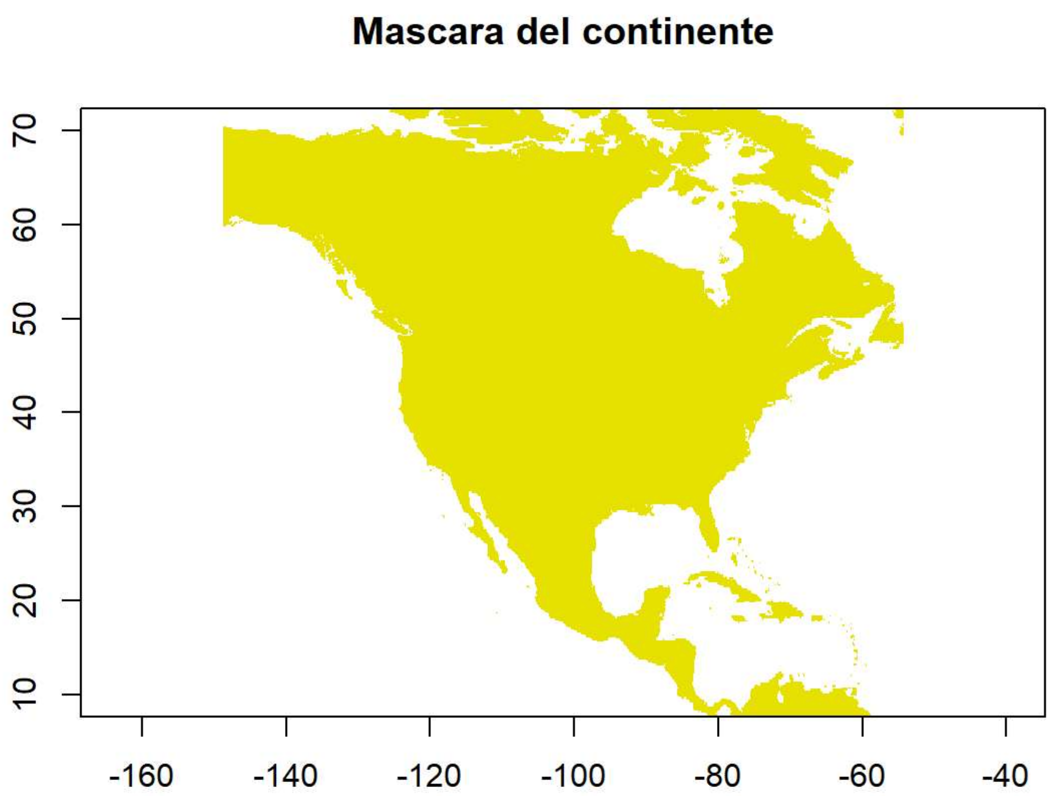

```
plot(bio_vars, c(31:36), col = grey(1:100/100), nc=3)
```




```
plot(bio_vars, c(37:40), col = grey(1:100/100), nc=3)
```



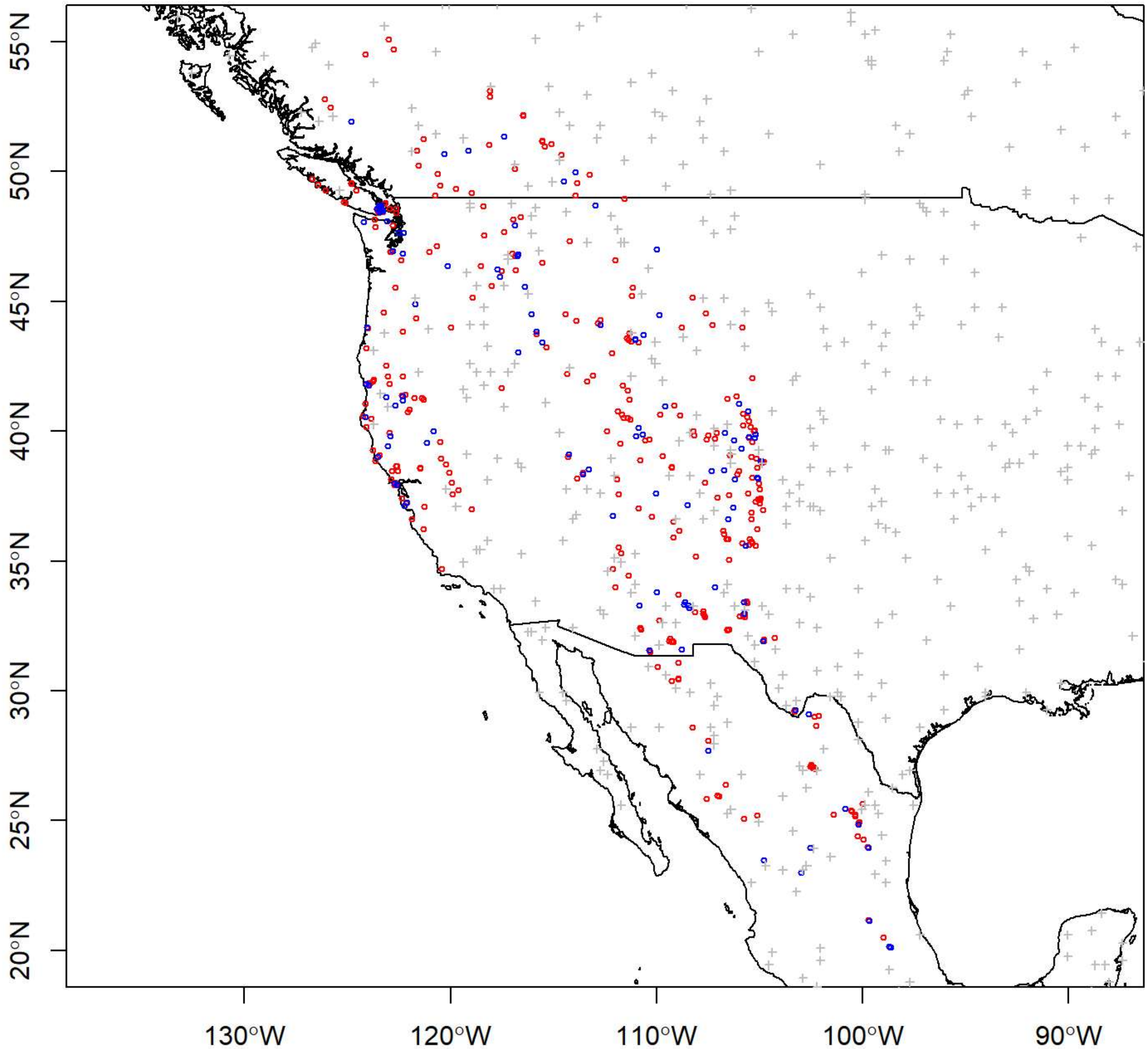

```
mask_reclass_table <- matrix(c(-Inf, 0, NA, 0, Inf, 1), ncol=3, byrow=TRUE)
bio_vars_mask <- reclassify(subset(bio_vars, 12), mask_reclass_table)
plot(bio_vars_mask, legend = FALSE, main = "Mascara del continente")
```



Definir conjuntos de puntos de entrenamiento y de control

```
set.seed(1)

fold <- kfold(ocurrencias, k = 5)
ocurrencias_entrenamiento <- ocurrencias[ fold != 1, ]
ocurrencias_control <- ocurrencias[ fold == 1, ]

plot(mundo, axes = TRUE, xlim = c(-130,-95), ylim = c(20,55),
     main = "Puntos de entrenamiento y de control de Pseudotsuga menziesii")
points(ocurrencias_entrenamiento$decimalLongitude, ocurrencias_entrenamiento$decimalLatitude,
       cex = 0.5, col = "red")
points(ocurrencias_control$decimalLongitude, ocurrencias_control$decimalLatitude,
       cex = 0.5, col = "blue")

puntos_aleatorios_fondo <- randomPoints(bio_vars_mask, 1000)
points(puntos_aleatorios_fondo, col = "gray", cex = 0.5, pch = 3)
```

Puntos de entrenamiento y de control de Pseudotsuga menziesii



Realizar muestreo de valores ambientales

```
x_presencia <- extract(subset(bio_vars, 1:35), ocurrencias_entrenamiento[,2:3])
x_control <- extract(subset(bio_vars, 1:35), ocurrencias_control[,2:3])
x_fondo <- extract(subset(bio_vars, 1:35), puntos_aleatorios_fondo)

# generar vector de presincias (1) y valores del fondo (0)
y <- c(rep(1, dim(ocurrencias_entrenamiento)[1]),
       rep(0, dim(puntos_aleatorios_fondo)[1]))
x <- rbind(x_presencia, x_fondo)

specie_entrenamiento <- as.data.frame(cbind(x,y))
dim(specie_entrenamiento)
```

```
## [1] 1465   36
```

```
#str(specie_entrenamiento)
head(specie_entrenamiento)
```

```
##          b1       b2        b3          b4      b5       b6       b7        b8
## 1  9.493113 17.57683 0.5020993 0.022678930 27.5600 -7.44667 35.00667 17.362181
## 2 14.580640 16.90603 0.4689173 0.024646200 32.6800 -3.37333 36.05333 23.088070
## 3 11.342480  8.07449 0.5507676 0.008065776 18.9871  4.32667 14.66043  8.874005
## 4 10.440200 15.89849 0.4733104 0.022967471 27.4700 -6.12000 33.59000 18.321030
## 5 14.580640 16.90603 0.4689173 0.024646200 32.6800 -3.37333 36.05333 23.088070
## 6 13.854520 16.09004 0.4620034 0.024033669 31.3800 -3.44667 34.82667 22.054529
##          b9      b10      b11       b12     b13     b14       b15      b16
## 1  9.062980 17.84807 1.554323  468.9998 22.5160 2.38946 0.7022920 247.4439
## 2 12.690760 23.67258 5.622020  293.0000 14.0862 1.46639 0.7014532 153.8621
## 3 14.205380 14.37337 8.661698 1447.0000 60.4494 1.20713 0.7757894 730.8978
## 4  7.572446 18.83243 2.264138  426.0001 20.8721 2.75627 0.7148855 227.7247
## 5 12.690760 23.67258 5.622020  293.0000 14.0862 1.46639 0.7014532 153.8621
## 6 12.045030 22.67207 5.127740  352.0000 15.4698 2.22127 0.6119072 172.8021
##        b17       b18       b19      b20     b21      b22       b23       b24
## 1 45.12693 221.94791  80.18539 180.1977 270.841 100.3830 0.2983824 217.79880
## 2 26.08244 139.76450  45.53280 184.0221 273.314 103.8580 0.2917426 221.10310
## 3 38.56227  57.96872 714.88000 135.4905 219.320  57.0180 0.4175186  64.13944
## 4 52.74678 216.56461  58.93639 177.0775 261.608  99.2535 0.2963731 215.73830
## 5 26.08244 139.76450  45.53280 184.0221 273.314 103.8580 0.2917426 221.10310
## 6 40.02862 158.74860  58.29061 179.5582 264.855 101.2660 0.2925339 217.18370
##        b25      b26       b27       b28      b29      b30       b31       b32
## 1 215.1306 234.0828 113.15160 0.4115424 0.607546 0.221662 0.2691783 0.5494443
## 2 202.9901 237.6931 116.98480 0.2870778 0.392857 0.159496 0.2450126 0.3671726
## 3 199.7792 185.6432  70.07321 0.9549514 1.503062 0.280562 0.4689641 1.4582940
## 4 186.1920 231.5430 111.77560 0.3842115 0.565269 0.257073 0.2249405 0.5118567
## 5 202.9901 237.6931 116.98480 0.2870778 0.392857 0.159496 0.2450126 0.3671726
## 6 197.8496 233.1818 114.07550 0.3291340 0.426847 0.200007 0.2102104 0.4037451
##         b33       b34       b35 y
## 1 0.2555285 0.4615018 0.4419149 1
## 2 0.1831211 0.3126832 0.3123895 1
## 3 0.3352059 0.3365532 1.4436181 1
## 4 0.2858154 0.4620836 0.3686208 1
## 5 0.1831211 0.3126832 0.3123895 1
## 6 0.2239663 0.3491877 0.3588941 1
```

```
y_c <- c(rep(1, dim(ocurrencias_control)[1]),
       rep(0, dim(puntos_aleatorios_fondo)[1]))
x_c <- rbind(x_control, x_fondo)

specie_control <- as.data.frame(cbind(x_c,y_c))
```

Construir el modelo MAXENT

```
jar <- paste(system.file(package='dismo'), '/java/maxent.jar', sep='')

if (file.exists(jar)) {

  ## modelo construido con datos en los resultados de muestreo (subconjunto entrenamiento)
  modelo_maxent <- maxent(
                        x = specie_entrenamiento[1:35],
                        p = specie_entrenamiento[36],
                        silent = FALSE,
                        args=c(
                          'removeDuplicates=TRUE',
                          'jackknife=TRUE',
                          'responsecurves=TRUE',
                          'threads=2',
                          'linear=TRUE',
                          'threshold=FALSE',
                          'quadratic=TRUE',
                          'hinge=FALSE',
                          'product=FALSE',
                          'maximumiterations=2000'
                        ))
    summary(modelo_maxent)
    saveRDS(modelo_maxent, "modelo_maxent_Pseudotsuga_menziesii.rds")

} else {
  print('maxent.jar no disponible')
}
```

Realizar evaluación del modelo

```
evaluacion_e <- evaluate(model = modelo_maxent,
              p = specie_entrenamiento[specie_entrenamiento$y == 1,1:35],
              a = specie_entrenamiento[specie_entrenamiento$y == 0,1:35])
evaluacion_e
```

```
## class          : ModelEvaluation
## n presences    : 465
## n absences     : 1000
## AUC            : 0.9501914
## cor            : 0.7865341
## max TPR+TNR at : 0.3454397
```

```
evaluacion_c <- evaluate(model = modelo_maxent,
              p = specie_control[specie_control$y == 1,1:35],
              a = specie_control[specie_control$y == 0,1:35]
              )
evaluacion_c
```

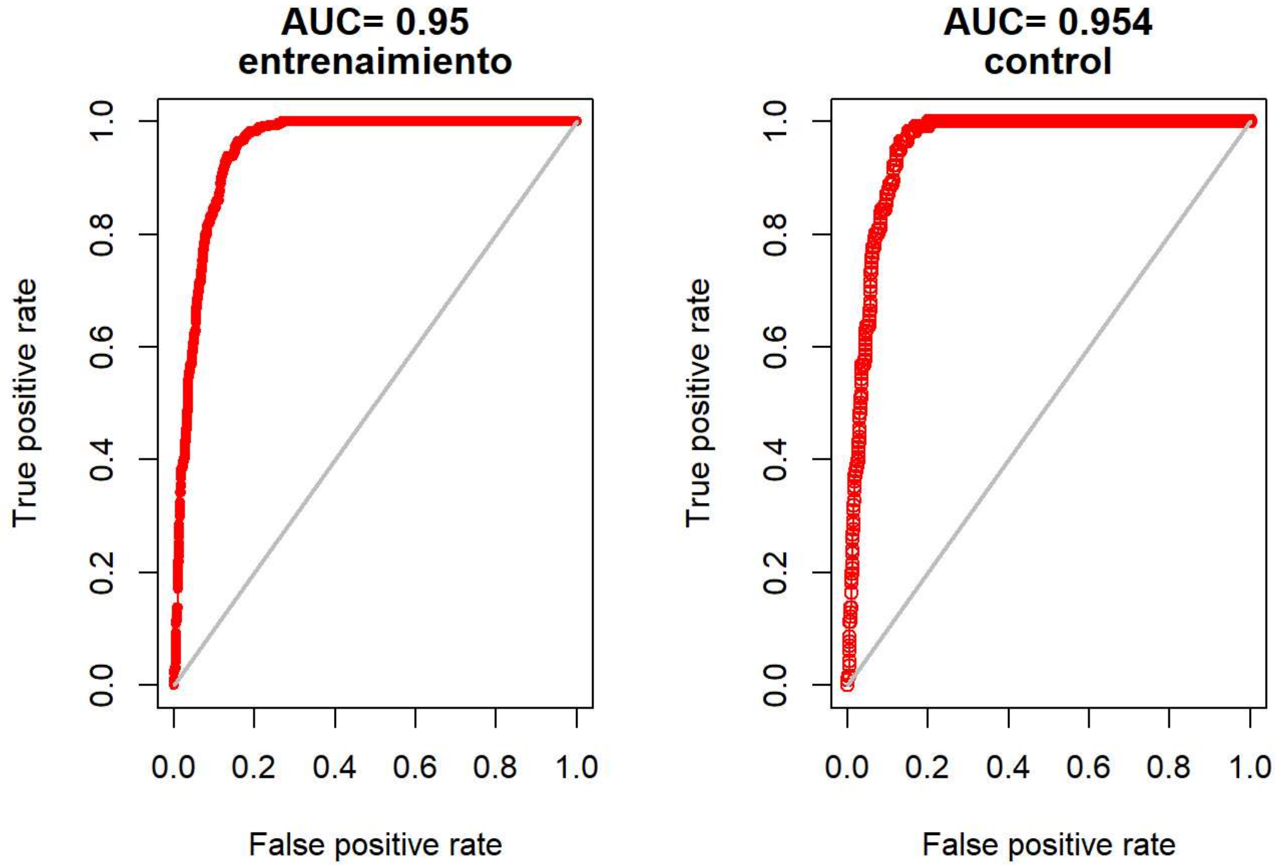
```
## class          : ModelEvaluation
## n presences    : 116
## n absences     : 1000
## AUC            : 0.9543922
## cor            : 0.633321
## max TPR+TNR at : 0.4020695
```

```
threshold(evaluacion_c)
```

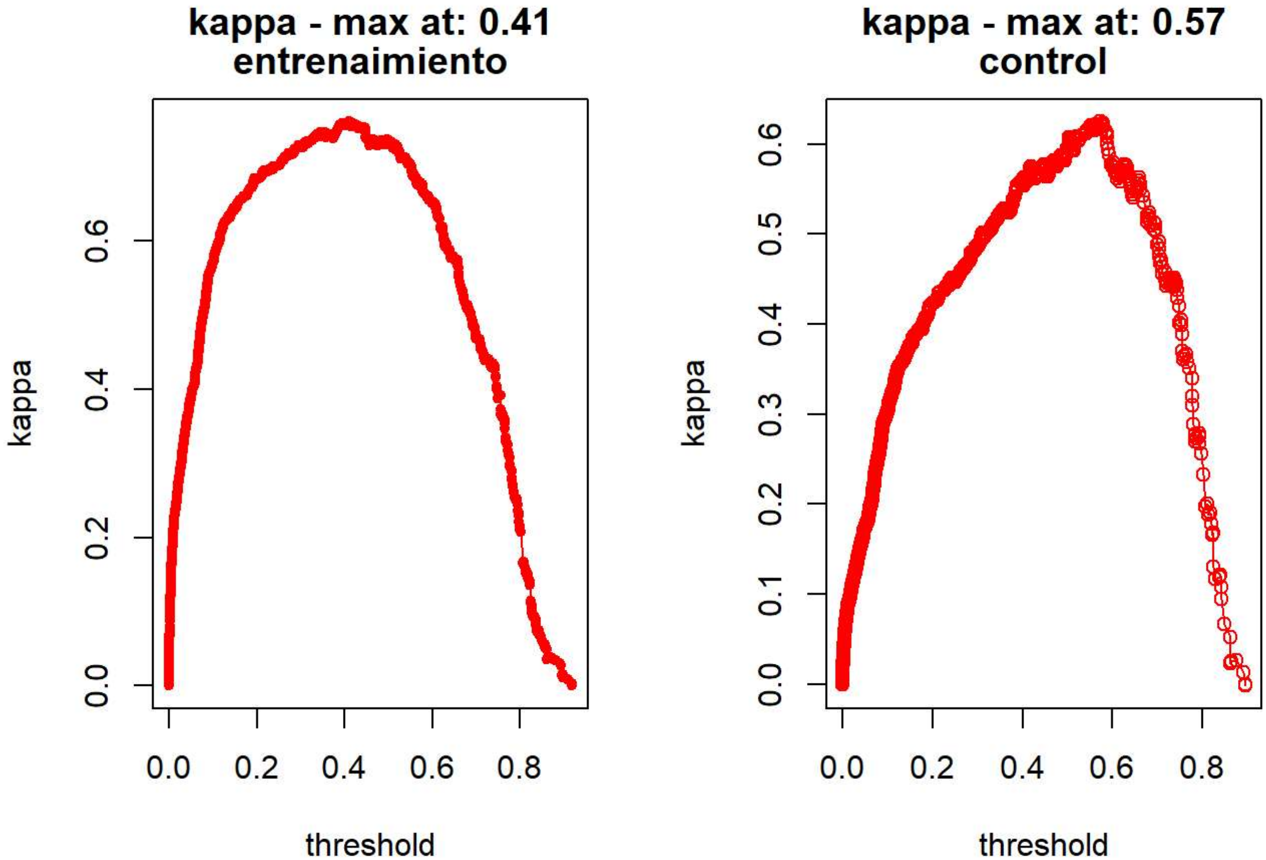
```
##                kappa spec_sens no_omission prevalence equal_sens_spec
## thresholds 0.5714061 0.4020695   0.2431406  0.1042252       0.4571496
##            sensitivity
## thresholds    0.451235
```

Presentar graficas de evaluación del modelo

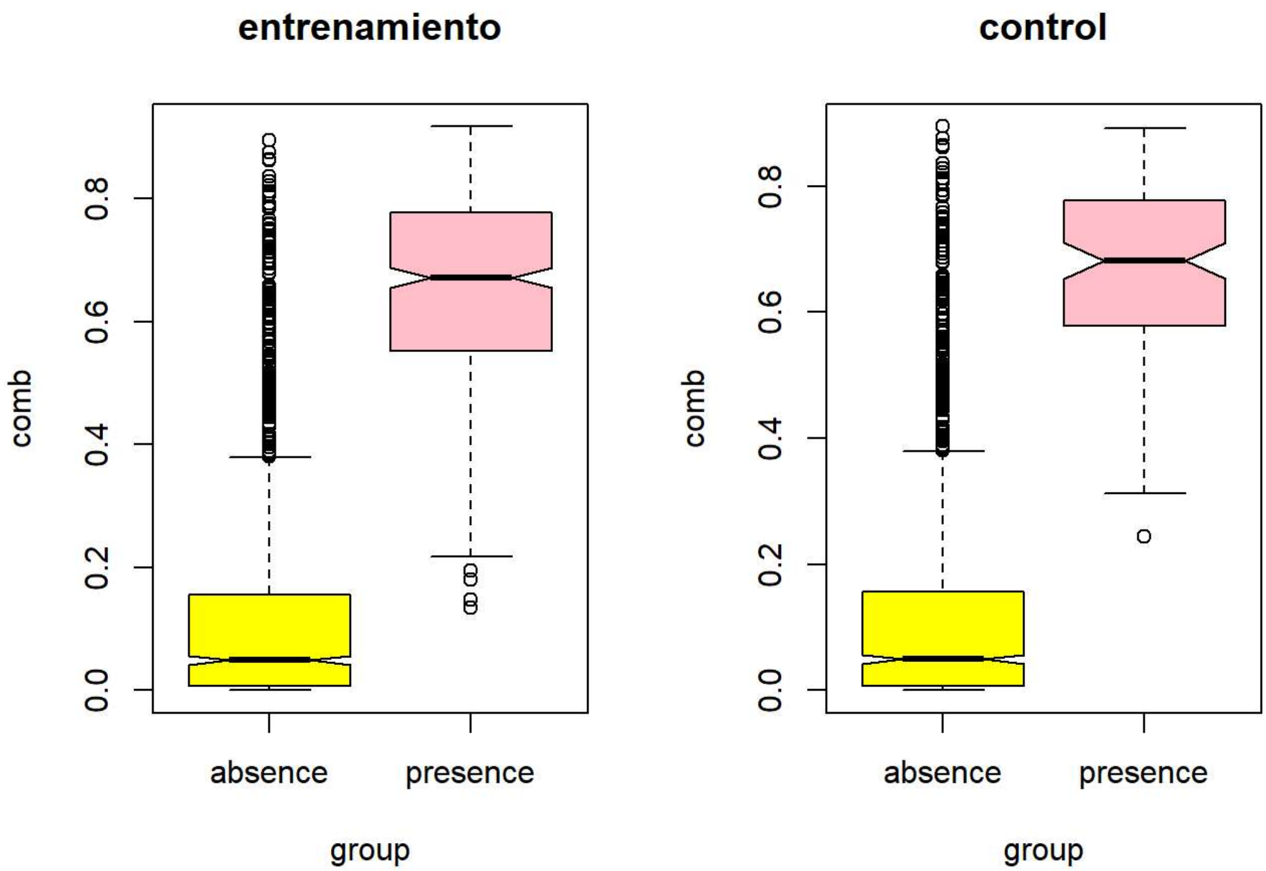
```
par(mfcol = c(1,2))

plot(evaluacion_e, 'ROC', pch=20)
title(c(" "," ","entrenaimiento"))
plot(evaluacion_c, 'ROC')
title(c(" "," ","control"))
```



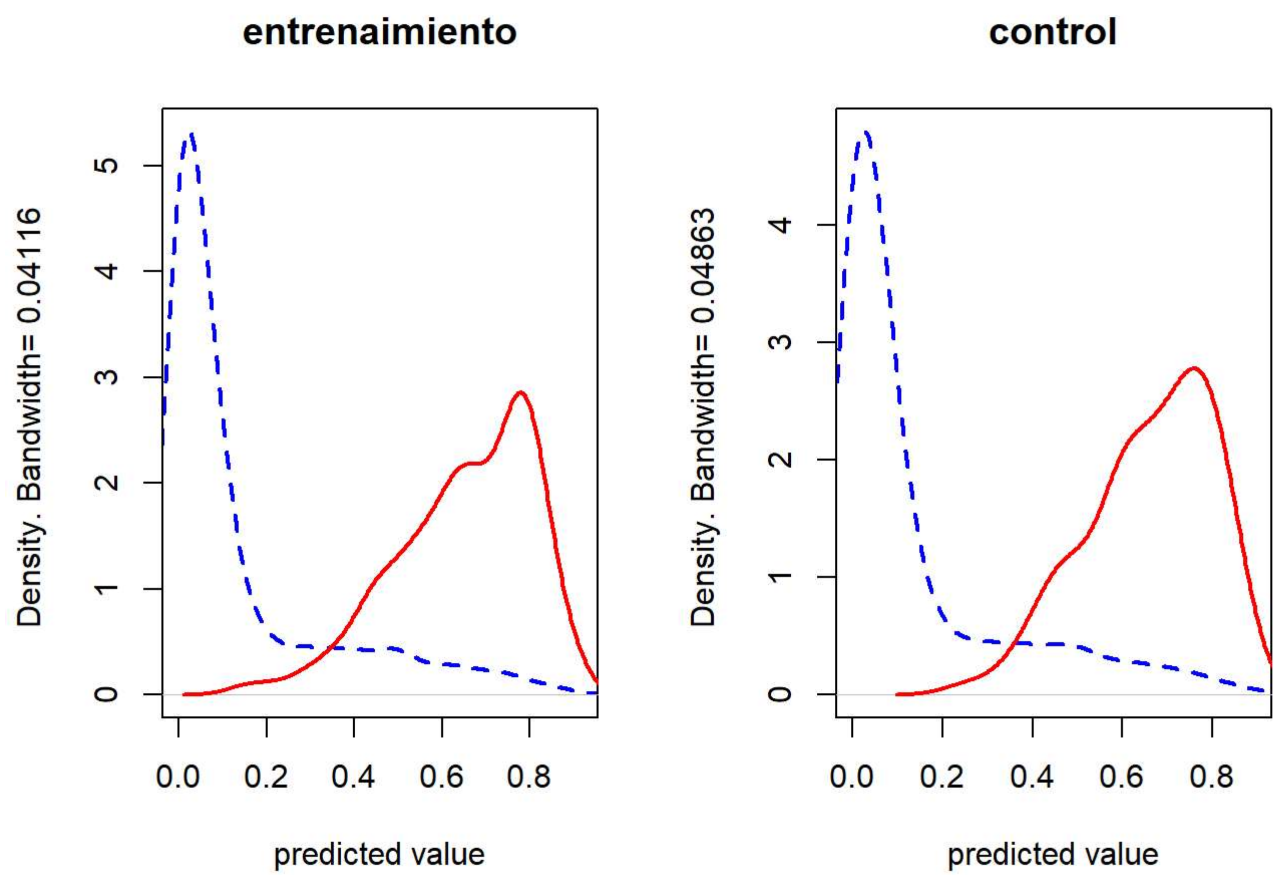

```
plot(evaluacion_e, 'kappa', pch=20)
title(c(" "," ","entrenaimiento"))
plot(evaluacion_c, 'kappa')
title(c(" "," ","control"))
```




```
boxplot(evaluacion_e, col=c('yellow','pink'), notch=TRUE,
        main = "entrenamiento")
boxplot(evaluacion_c, col=c('yellow','pink'), notch=TRUE,
        main = "control")
```

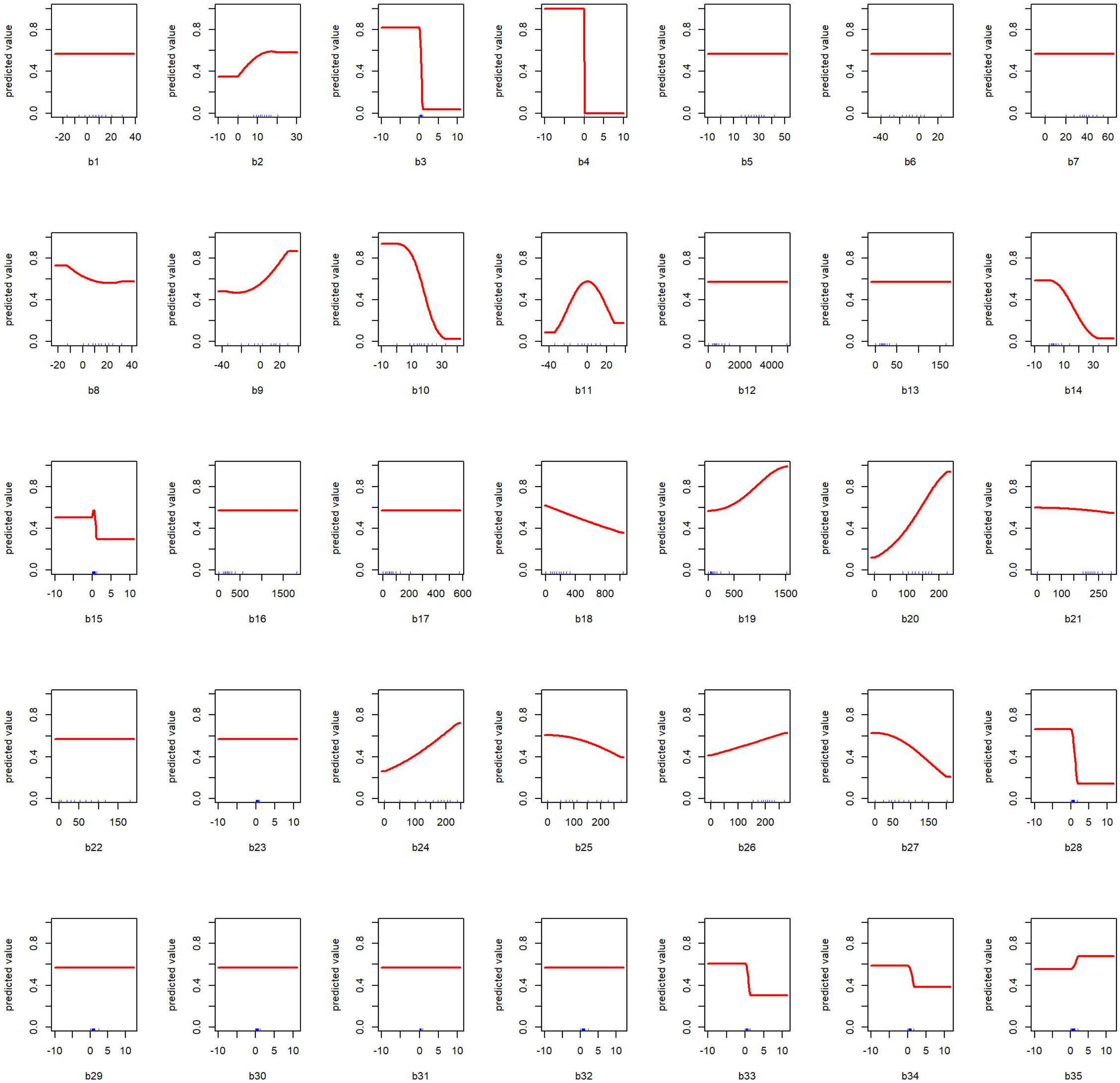



```
density(evaluacion_e)
title("entrenaimiento")
density(evaluacion_c)
title("control")
```



Presentar graficas de contribución de variables

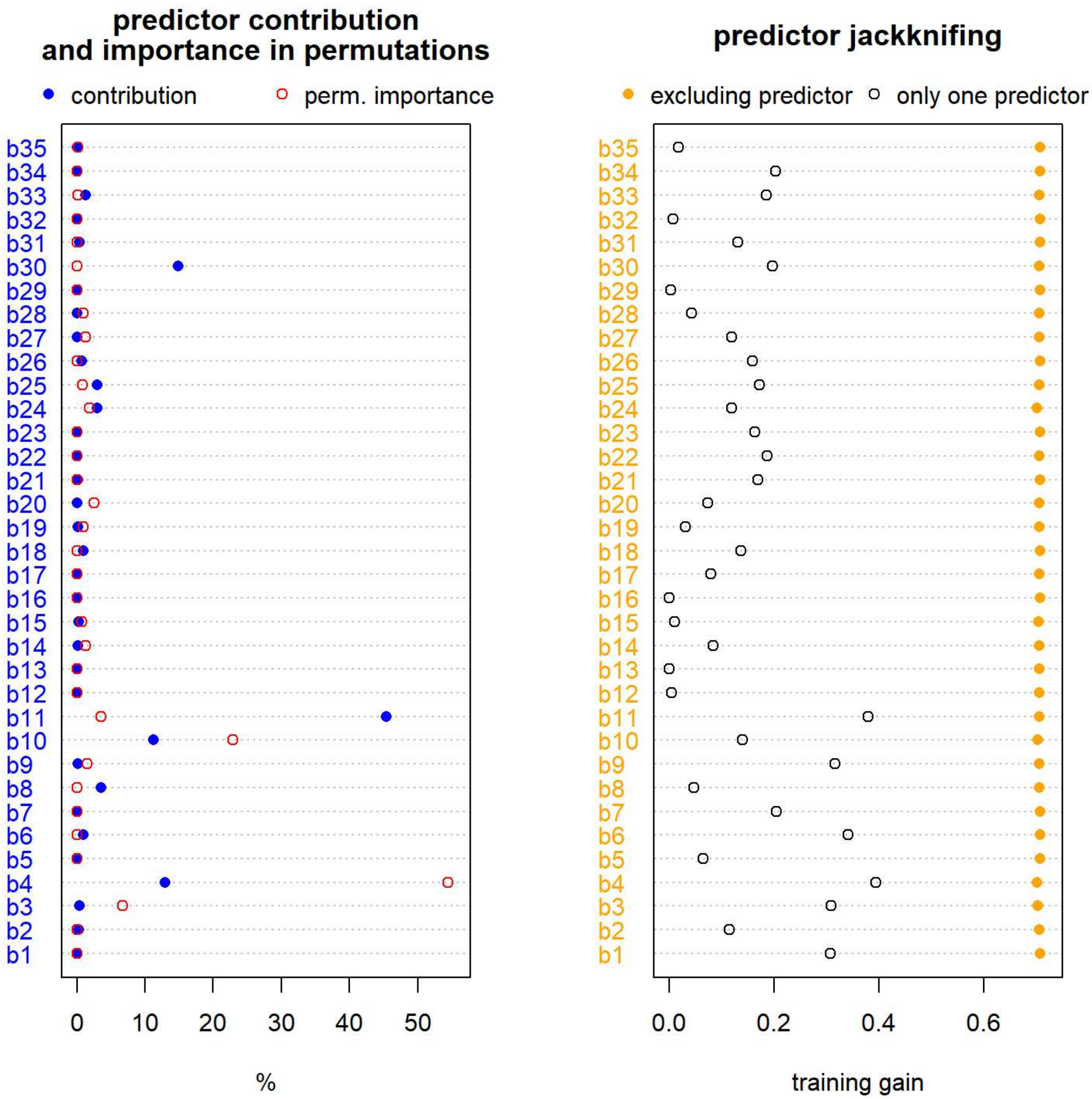
```
response(modelo_maxent)
```




```
#modelo_maxent@results
```

```r
maxent.select.contribution <- function (m, n) {
  ## parametros: m - objeto del model maxent, n - vector de nombres de variables raster
  ## Note: with jackknifing - in order to get variables 3 and 4
  resultados <- m@results
  tabla_resultados <- data.frame(t(rep(NA,5)))
  names(tabla_resultados) <- c('variable','contribution','permutation.importance',
                               'gain.without','gain.only')
  for (i in 1:length(n)) {
    var1_name <- paste0(n[i],'.contribution')
    var1_value <- resultados[var1_name,1]
    var2_name <- paste0(n[i],'.permutation.importance')
    var2_value <- resultados[var2_name,1]
    var3_name <- paste0('Training.gain.without.',n[i])
    var3_value <- resultados[var3_name,1]
    var4_name <- paste0('Training.gain.with.only.',n[i])
    var4_value <- resultados[var4_name,1]
    tabla_resultados <- rbind(tabla_resultados,
        c(n[i],var1_value,var2_value,var3_value,var4_value))
  }
  tabla_resultados <- tabla_resultados[-1,]
  return(tabla_resultados)
}
```

```
nombres_variables <- names(subset(bio_vars, 1:35))
contribuciones_variables <- maxent.select.contribution(modelo_maxent,
                                                      nombres_variables)
contr_max <- max(as.numeric(c(contribuciones_variables[,2],
                              contribuciones_variables[,3])))
gain_max <- max(as.numeric(c(contribuciones_variables[,4],
                              contribuciones_variables[,5])))

par(mfcol = c(1,2), mar = c(5.1, 4.1, 6.1, 2.1))

dotchart(as.numeric(contribuciones_variables[,2]),
          col='blue', pch=16,
          labels=contribuciones_variables[,1],
          xlim=c(0, 1.02 * contr_max),
          main=c('predictor contribution','and importance in permutations'),
          xlab = "%"
         )

points(as.numeric(contribuciones_variables[,3]),
       seq(1:35),
       col='red', pch=1)

legend("top", inset=c(0,-0.07), xpd = TRUE,
       ncol = 2, bty = "n",
       legend = c("contribution","perm. importance"),
       pch = c(16,1), col = c("blue","red"))

dotchart(as.numeric(contribuciones_variables[,4]),
          col = 'orange', pch=16,
          labels = contribuciones_variables[,1],
          xlim = c(0, 1.02 * gain_max),
          main = 'predictor jackknifing',
          xlab = 'training gain')

points(as.numeric(contribuciones_variables[,5]),
       seq(1:35),
       col='black', pch=1)

legend("top", inset=c(0,-0.07), xpd = TRUE,
       ncol = 2, bty = "n",
       legend = c("excluding predictor","only one predictor"),
       pch = c(16,1), col = c("orange","black"))
```

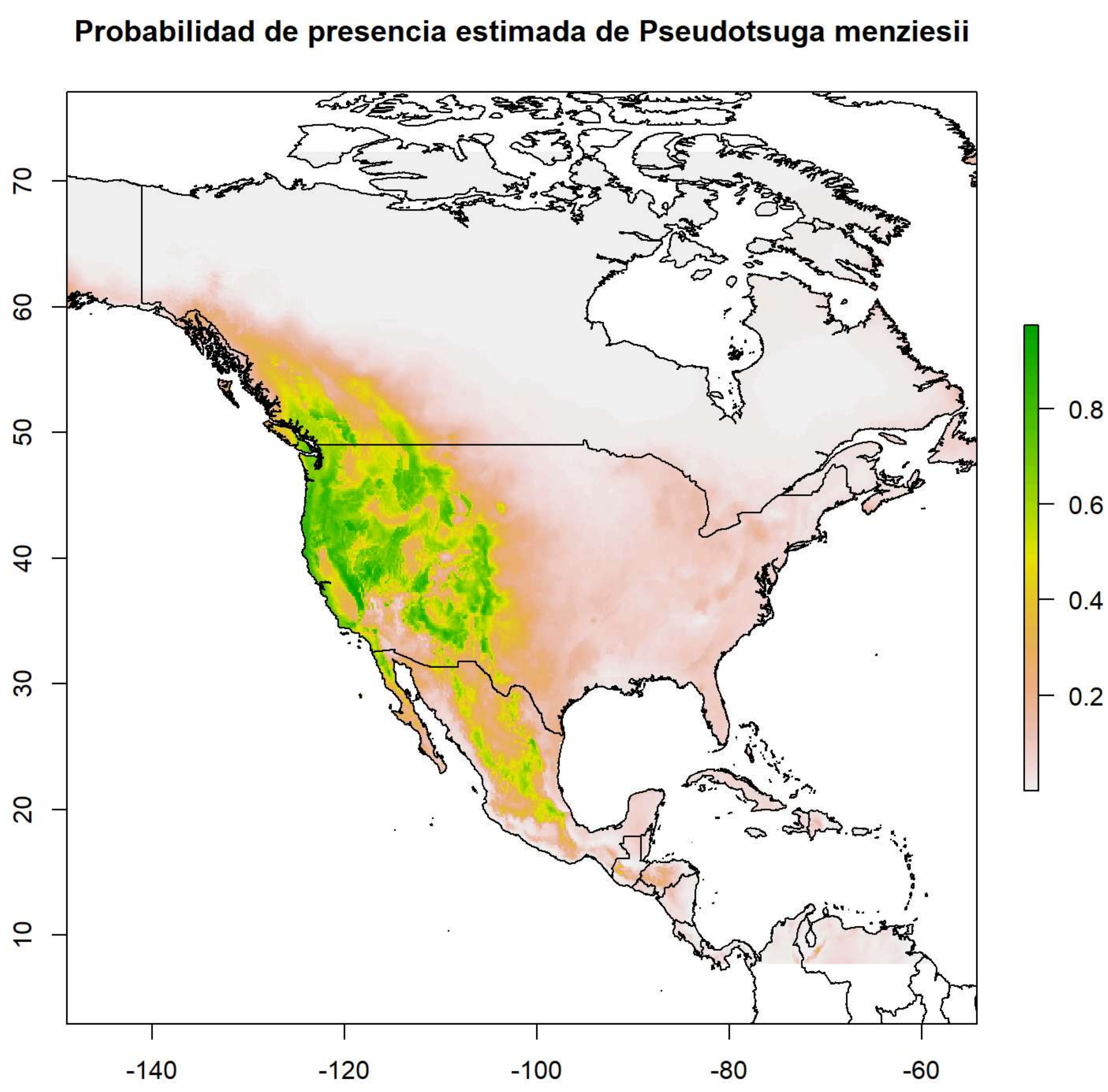


Realizar prediccion de idoneidad de habitat y clasificación binaria

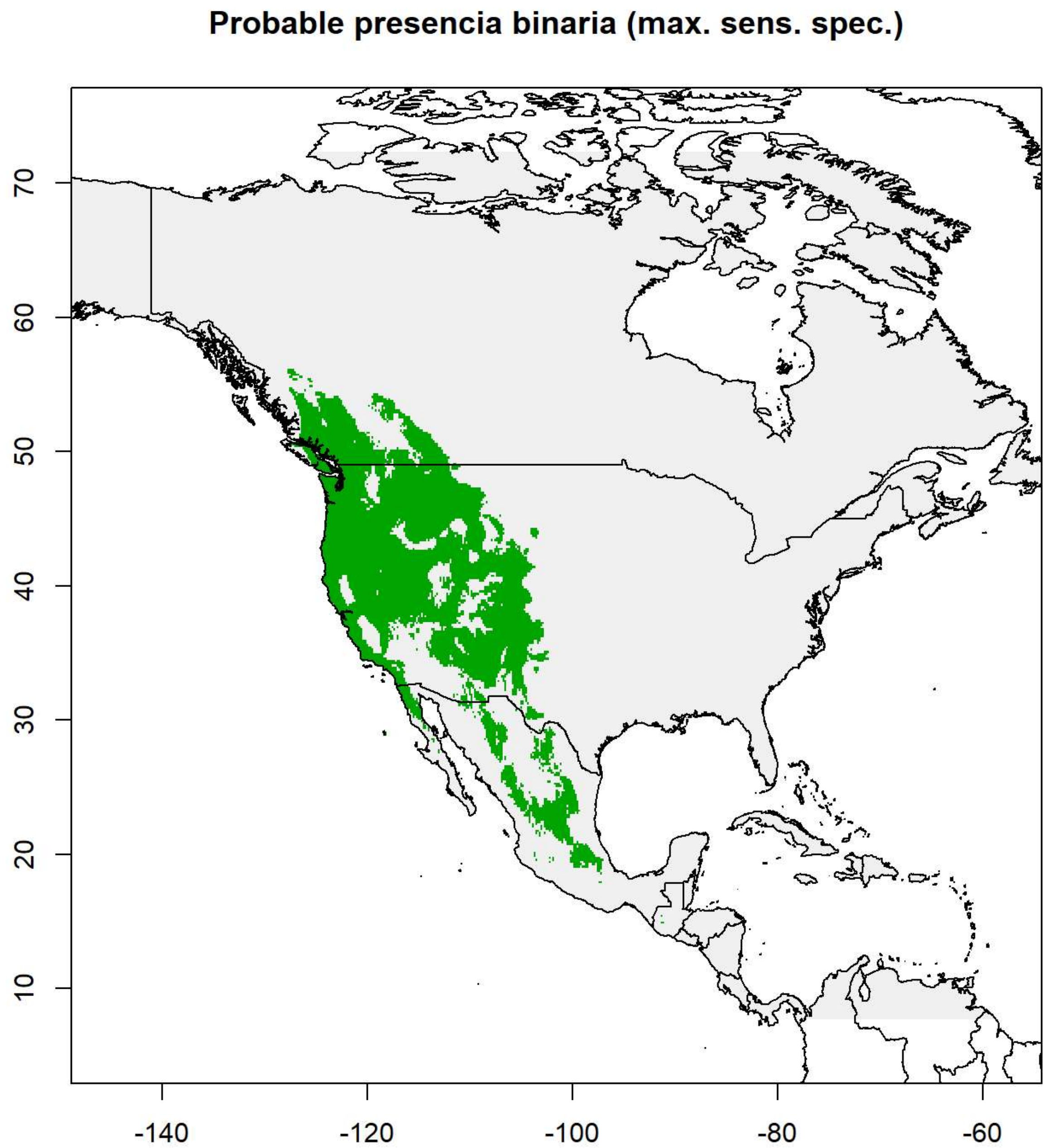
```
prediccion <- predict(subset(bio_vars, 1:35), modelo_maxent, progress='window')
```

```
## Loading required namespace: tcltk
```

```
prediccion <- prediccion * bio_vars_mask
plot(prediccion, main = "Probabilidad de presencia estimada de Pseudotsuga menziesii")
plot(mundo, add = TRUE)
```




```
especie_reclass_table <- matrix(c(-Inf, threshold(evaluacion_c)$spec_sens, 0,
                                 threshold(evaluacion_c)$spec_sens, Inf, 1),
                               ncol=3, byrow=TRUE)
prediccion_presencia <- reclassify(prediccion, especie_reclass_table)
plot(prediccion_presencia, legend = FALSE,
     main = "Probable presencia binaria (max. sens. spec.)")
plot(mundo, add = TRUE)
```



Guadrar rasters

```
writeRaster(prediccion,
             filename = "probabilidad_presencia_Pseudotsuga_menziesii.tif",
            format="GTiff", datatype = "FLT4S", overwrite = TRUE)

writeRaster(prediccion_presencia,
             filename = "clasificacion_binaria_Pseudotsuga_menziesii.tif",
            format="GTiff", datatype = "INT2S", overwrite = TRUE)
```